REPORT ON THE CRYTO BOT

Binance Futures Order Bot - Project Report

1. Introduction

This project implements an automated trading bot for Binance USDT-M Futures, supporting core order types such as market and limit orders, as well as advanced order strategies including OCO and TWAP. The bot uses the official Binance Futures Testnet API to provide a safe environment for testing trading logic, with comprehensive input validation, error handling, and structured logging.

---

2. System Design and Architecture

The bot is modularly designed with separate Python scripts handling different order types inside the src/ directory. Core orders are implemented in market_orders.py and limit_orders.py. Advanced order logic, such as OCO and TWAP implementations, resides in src/advanced/.

The bot integrates with Binance Futures Testnet through the python-binance library, setting the API base to the testnet endpoint. Logging is handled centrally, outputting detailed order and error information into bot.log at the project root, enabling troubleshooting and audit.

---

3. Implementation Details

Market Orders

Users provide trading symbol, side (buy/sell), and quantity. The bot validates inputs and submits a market order through the Binance API. Confirmation details are printed and logged.

Limit Orders

In addition to market order inputs, limit orders require a user-entered price. The bot ensures the price is a valid float before placing a limit order with GTC time-in-force.

Advanced Orders

OCO Orders

Since Binance Futures does not natively support OCO, the bot creates a take profit limit order and a stop limit order independently. Manual cancellation of one order upon the other's execution requires further event handling, which is noted as a limitation.

TWAP

The TWAP strategy splits a large order into smaller market orders executed over equal intervals. Parameters include total quantity, number of slices, and interval duration in seconds.

Logging and Error Handling

All actions, including order placement and exceptions, are logged with timestamps, facilitating easy review of trading activity and issues.

---

4. Testing and Results

The bot was tested exclusively in Binance Futures Testnet to ensure zero financial exposure risks. Market and limit orders executed as expected, with the console confirming successful placement and bot.log recording the details.

OCO and TWAP advanced orders demonstrated correct API requests and logging. Due to the complexity of real-time order cancellation for OCO, full cycle automation remains an enhancement objective.

---

5. Challenges and Limitations

The primary challenge was managing complex order types such as OCO due to Binance Futures API constraints. Ensuring robust input validation and handling various error scenarios required careful design. Future improvements include implementing live order monitoring for advanced strategies and secure management of API credentials.

---

6. Conclusion

This project developed a functional Binance Futures trading bot with clean modular architecture capable of handling essential order types along with advanced strategy foundations. The bot's design and comprehensive logging facilitate further extension and real-world deployment readiness pending risk testing.

Source code:

```python
from binance.client import Client
import logging
class BinanceFuturesBot:
    def __init__(self, api_key, api_secret, testnet=True):
        self.client = Client(api_key, api_secret, testnet=testnet)
        self.client.FUTURES_URL = 'https://testnet.binancefuture.com/fapi' if testnet else self.client.FUTURES_URL
        logging.basicConfig(filename='bot.log', level=logging.INFO,
                            format='%(asctime)s : %(levelname)s : %(message)s')
    def place_market_order(self, symbol, side, quantity):
        try:
            order = self.client.futures_create_order(
                symbol=symbol,
                side=side,
                type='MARKET',
                quantity=quantity
            )
            logging.info(f"Market order placed: {order}")
            print(f"Market order placed: {order}")
            return order
        except Exception as e:
            logging.error(f"Market order failed: {e}")
            print(f"Market order failed: {e}")
            return None
    def place_limit_order(self, symbol, side, quantity, price):
        try:
            order = self.client.futures_create_order(
                symbol=symbol,
```

The Binance Futures Order Bot project is a command-line interface (CLI) trading bot designed to automate the placement of various types of orders on the Binance USDT-M Futures Testnet. The bot supports essential order types such as market orders and limit orders, as well as advanced strategies like One-Cancels-the-Other (OCO) and Time-Weighted Average Price (TWAP) orders.

The project is structured modularly, with different scripts for each order type, promoting clarity and ease of maintenance. It uses the official Binance API client configured to interact with the Binance Futures Testnet environment, ensuring safe and risk-free experimentation. Input validation and exception handling are integral parts of the bot, allowing users to input trading symbols, order sides, quantities, prices, and stop prices interactively. The bot provides immediate console feedback on order placements and errors, while also logging detailed transaction records in a structured log file for auditing and debugging purposes.

This approach enables traders and developers to automate common trading tasks, test different strategies, and gain practical experience with Binance Futures API workflows. The bot serves as a foundational platform that can be extended with more complex order types,

better risk management, and integration of market data analytics, making it a versatile tool for algorithmic trading development on Binance Futures.

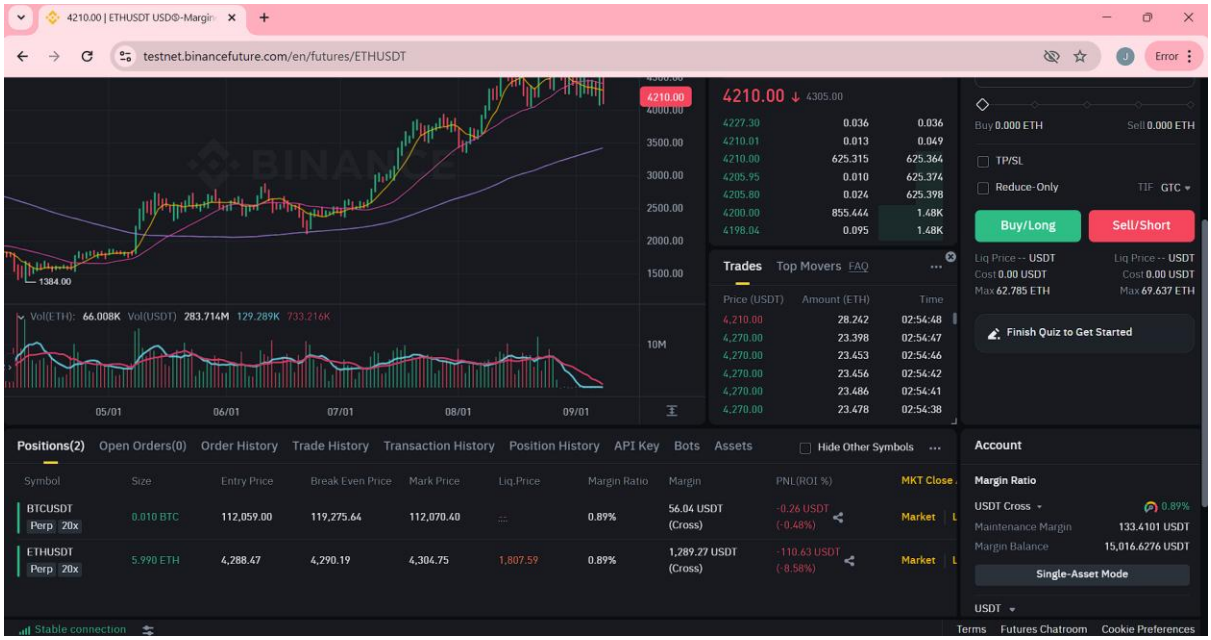
```
                side=side,
                type='LIMIT',
                quantity=quantity,
                price=price,
                timeInForce='GTC'
            )
            logging.info(f"Limit order placed: {order}")
            print(f"Limit order placed: {order}")
            return order
        except Exception as e:
            logging.error(f"Limit order failed: {e}")
            print(f"Limit order failed: {e}")
            return None
    def place_stop_market_order(self, symbol, side, quantity, stop_price):
        try:
            order = self.client.futures_create_order(
                symbol=symbol,
                side=side,
                type='STOP_MARKET',
                quantity=quantity,
                stopPrice=stop_price,
                timeInForce='GTC'
            )
            logging.info(f"Stop market order placed: {order}")
            print(f"Stop market order placed: {order}")
```

```
        except Exception as e:
            logging.error(f"Stop market order failed: {e}")
            print(f"Stop market order failed: {e}")
            return None

def main():
    print("Welcome to Binance Futures Bot")
    print("Order types: MARKET, LIMIT, STOP_MARKET")
    order_type = input("Enter order type: ").strip().upper()
    symbol = input("Enter trading symbol (e.g., BTCUSDT): ").strip().upper()
    side = input("Enter side (BUY or SELL): ").strip().upper()

    quantity_input = input("Enter quantity: ").strip()
    try:
        quantity = float(quantity_input)
    except ValueError:
        print("Invalid quantity. Must be a number.")
        return

    API_KEY = "6fb8c22049dcaf1906093a75f421029449969dcb85391df8e9271d8fe1e3b227"
    API_SECRET = "00cde179d8253511429a0207defc11712be7fa729c124c8999776c6c93eabcfb"

    bot = BinanceFuturesBot(API_KEY, API_SECRET, testnet=True)

    if order_type == "MARKET":
```

```python
    bot = BinanceFuturesBot(API_KEY, API_SECRET, testnet=True)

    if order_type == "MARKET":
        bot.place_market_order(symbol, side, quantity)

    elif order_type == "LIMIT":
        price_input = input("Enter limit price: ").strip()
        try:
            price = float(price_input)
        except ValueError:
            print("Invalid price. Must be a number.")
            return
        bot.place_limit_order(symbol, side, quantity, price)

    elif order_type == "STOP_MARKET":
        stop_price_input = input("Enter stop price: ").strip()
        try:
            stop_price = float(stop_price_input)
        except ValueError:
            print("Invalid stop price. Must be a number.")
            return
        bot.place_stop_market_order(symbol, side, quantity, stop_price)
    else:
        print(f"Unsupported order type: {order_type}. Choose MARKET, LIMIT or STOP_MARKET.")
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Welcome to Binance Futures Bot
Order types: MARKET, LIMIT, STOP_MARKET
Enter order type: market
Enter trading symbol (e.g., BTCUSDT): ethusdt
Enter side (BUY or SELL): buy
Enter quantity: 5.99
Market order placed: {'orderId': 5006536626, 'symbol': 'ETHUSDT', 'status': 'NEW', 'clientOrderId': 'x-Cb7ytekJ925fba889e20cea4979b96', 'pr
ice': '0.00', 'avgPrice': '0.00', 'origQty': '5.990', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC
', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType'
: 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTill
Date': 0, 'updateTime': 1757366582418}
PS C:\Users\HP\Desktop>
```


4210.00 | ETHUSDT USD©-Margin
testnet.binancefuture.com/en/futures/ETHUSDT
Error
4210.00
4305.00
Buy 0.000 ETH
Sell 0.000 ETH
TP/SL
Reduce-Only
TIF GTC
Buy/Long
Sell/Short
Liq Price -- USDT
Cost 0.00 USDT
Max 62.785 ETH
Liq Price -- USDT
Cost 0.00 USDT
Max 69.637 ETH
Finish Quiz to Get Started
Trades
Top Movers
FAQ
Price (USDT)
Amount (ETH)
Time
Positions(2)
Open Orders(0)
Order History
Trade History
Transaction History
Position History
API Key
Bots
Assets
Hide Other Symbols
Account
Margin Ratio
USDT Cross
0.89%
Maintenance Margin
133.4101 USDT
Margin Balance
15,016.6276 USDT
Single-Asset Mode
USDT
Symbol
Size
Entry Price
Break Even Price
Mark Price
Liq.Price
Margin Ratio
Margin
PNL(ROI %)
MKT Close
BTCUSDT
Perp 20x
0.010 BTC
112,059.00
119,275.64
112,070.40
0.89%
56.04 USDT
(Cross)
-0.26 USDT
(-0.48%)
Market
ETHUSDT
Perp 20x
5.990 ETH
4,288.47
4,290.19
4,304.75
1,807.59
0.89%
1,289.27 USDT
(Cross)
-110.63 USDT
(-8.58%)
Market
Stable connection
Terms
Futures Chatroom
Cookie Preferences